

Unit 5 - Shiny App Development in R

Contents

1. Introduction to Shiny	1
2. Basic UI Components	2
3. Server Logic.....	4
4. Reactive Programming.....	4
5. Sample Datasets	6
6. Complete Example App.....	8
7. Best Practices	17
Quick Reference Card.....	19
Sample Data Files	20

1. Introduction to Shiny

What is Shiny?

Shiny is an R package that makes it easy to build interactive web applications straight from R. No HTML, CSS, or JavaScript knowledge required.

Basic Structure

Every Shiny app has two main components:

1. **UI (User Interface)** - Defines how the app looks
2. **Server** - Defines how the app works

Installation

```
```\r\ninstall.packages("shiny")\r\nlibrary(shiny)\r\n```\r\n
```

## 2. Basic UI Components

### Layout Functions

```
```r
  Basic layout
fluidPage()           Responsive layout
sidebarLayout()       Sidebar + main panel
tabsetPanel()         Tabbed interface
navBarPage()          Navigation bar

  Example layout
ui <- fluidPage(
  titlePanel("My App Title"),
  sidebarLayout(
    sidebarPanel(
      Inputs go here
    ),
    mainPanel(
      Outputs go here
    )
  )
)
```
```

### Input Widgets

```
```r
  Text input
```

```
textInput("id", "Label", "Default value")

Numeric input
numericInput("id", "Label", value = 0, min = 0, max = 100)

Slider
sliderInput("id", "Label", min = 0, max = 100, value = 50)

Select dropdown
selectInput("id", "Label", choices = c("A", "B", "C"))

Checkbox
checkboxInput("id", "Label", value = FALSE)

Radio buttons
radioButtons("id", "Label", choices = c("Option1", "Option2"))

Date input
dateInput("id", "Label", value = Sys.Date())

File upload
fileInput("id", "Label", accept = c(".csv", ".txt"))
````
```

## Output Elements

```
````r
Text output
verbatimTextOutput("id")    Fixed-width text
textOutput("id")            Regular text

Table output
tableOutput("id")           Simple table
dataTableOutput("id")       Interactive table

Plot output
plotOutput("id", width = "100%", height = "400px")

Image output
imageOutput("id")
````
```

### 3. Server Logic

Basic Server Function

```
```r
server <- function(input, output, session) {
  Reactive calculations go here
  Output objects are created here
}
```
```

Render Functions

```
```r
# Render text
output$text <- renderText({
  paste("You entered:", input$userText)
})

# Render table
output$table <- renderTable({
  head(mtcars, input$rows)
})

# Render plot
output$plot <- renderPlot({
  hist(rnorm(input$n), main = "Random Histogram")
})

# Render data table
output$datatable <- renderDataTable({
  iris
})
```
```

### 4. Reactive Programming

Reactive Values

```
```r
```

```

server <- function(input, output, session) {
  Create reactive value
  values <- reactiveValues(counter = 0)

  Update reactive value
  observeEvent(input$button, {
    values$counter <- values$counter + 1
  })

  Display reactive value
  output$count <- renderText({
    paste("Count:", values$counter)
  })
}
```

```

## Reactive Expressions

```

```r
server <- function(input, output, session) {
  Create reactive calculation
  filtered_data <- reactive({
    subset(mtcars, mpg > input$mpg_threshold)
  })

  # Use reactive expression
  output$plot <- renderPlot({
    plot(filtered_data()$mpg, filtered_data()$hp)
  })
}
```

```

## Event Observers

```

```r
#Observe button click
observeEvent(input$goButton, {
  This code runs when button is clicked
  showNotification("Button clicked!", type = "message")
})

#Observe changes in input
observeEvent(input$slider, {
  Runs when slider changes
})
```

```

```
 cat("Slider value:", input$slider, "\n")
})
````
```

5. Sample Datasets

Built-in R Datasets

```
````r
1. mtcars - Motor Trend Car Road Tests
 32 observations, 11 variables
head(mtcars)
 mpg cyl disp hp drat wt qsec vs am gear carb
1 21.0 6 160 110 3.90 2.620 16.46 0 1 4 4

2. iris - Edgar Anderson's Iris Data
 150 observations, 5 variables
head(iris)
 Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1 5.1 3.5 1.4 0.2 setosa

3. airquality - New York Air Quality Measurements
 153 observations, 6 variables
head(airquality)
 Ozone Solar.R Wind Temp Month Day
1 41 190 7.4 67 5 1

4. diamonds - Diamond Prices
 53,940 observations, 10 variables
 (from ggplot2 package)
library(ggplot2)
head(diamonds)
````
```

Create Sample Datasets

```
````r
1. Sales Data
create_sales_data <- function(n = 100) {
 set.seed(123)
 data.frame(
 Date = seq.Date(from = as.Date("2024-01-01"),
```

```

 by = "day", length.out = n),
 Product = sample(c("A", "B", "C"), n, replace = TRUE),
 Region = sample(c("North", "South", "East", "West"), n,
replace = TRUE),
 Sales = round(runif(n, 100, 1000), 2),
 Customers = sample(10:100, n, replace = TRUE),
 stringsAsFactors = FALSE
)
}

```

```

sales_data <- create_sales_data()
head(sales_data)

```

## 2. Customer Data

```

create_customer_data <- function(n = 50) {
 set.seed(456)
 data.frame(
 CustomerID = 1001:(1000 + n),
 Age = sample(18:70, n, replace = TRUE),
 Gender = sample(c("Male", "Female"), n, replace = TRUE),
 Income = round(rnorm(n, mean = 50000, sd = 15000), 0),
 Score = round(runif(n, 0, 100), 1),
 Loyalty = sample(c("Low", "Medium", "High"), n, replace =
TRUE)
)
}

```

```

customer_data <- create_customer_data()

```

## 3. ML Classification Dataset

```

create_ml_dataset <- function(n = 200) {
 set.seed(789)
 data.frame(
 Feature1 = rnorm(n, mean = 10, sd = 2),
 Feature2 = rnorm(n, mean = 5, sd = 1.5),
 Feature3 = runif(n, 0, 1),
 Feature4 = rexp(n, rate = 0.5),
 Label = factor(sample(c("Class_A", "Class_B"), n,
replace = TRUE, prob = c(0.7, 0.3)))
)
}

```

```

ml_data <- create_ml_dataset()
...

```

## 6. Complete Example App

App 1: Data Explorer

```
```\r
# app.R
library(shiny)
library(ggplot2)

#UI
ui <- fluidPage(
  titlePanel("Data Explorer App"),

  sidebarLayout(
    sidebarPanel(
      width = 3,

      # Dataset selection
      selectInput("dataset", "Choose Dataset:",
                  choices = c("mtcars", "iris", "diamonds")),

      # Variable selection
      uiOutput("xvar"),
      uiOutput("yvar"),

      # Color by
      selectInput("color", "Color by:",
                  choices = NULL),

      # Plot type
      radioButtons("plotType", "Plot Type:",
                  choices = c("Scatter" = "scatter",
                              "Boxplot" = "box",
                              "Histogram" = "hist")),

      # Filter
      sliderInput("sampleSize", "Sample Size (%):",
                  min = 10, max = 100, value = 100, step = 10),

      #Action button
```



```

    actionButton("update", "Update Plot", class = "btn-
primary")
  ),

  mainPanel(
    width = 9,
    tabsetPanel(
      tabPanel("Plot",
        plotOutput("plot", height = "500px"),
        br(),
        downloadButton("downloadPlot", "Download
Plot")),

      tabPanel("Data",
        dataTableOutput("table")),

      tabPanel("Summary",
        verbatimTextOutput("summary"))
    )
  )
)

#Server
server <- function(input, output, session) {

  # Load selected dataset
  selected_data <- reactive({
    switch(input$dataset,
      "mtcars" = mtcars,
      "iris" = iris,
      "diamonds" = diamonds[sample(1:nrow(diamonds), 1000),
])
  })

  #Update variable choices based on dataset
  observe({
    data <- selected_data()
    num_vars <- names(data)[sapply(data, is.numeric)]
    all_vars <- names(data)

    updateSelectInput(session, "xvar", choices = num_vars)
    updateSelectInput(session, "yvar", choices = num_vars)
  })
}

```

```

    updateSelectInput(session, "color", choices = c("None",
all_vars))
  })

  #Filter data based on sample size
  filtered_data <- eventReactive(input$update, {
    data <- selected_data()
    n <- round(nrow(data)  input$sampleSize / 100)
    data[sample(1:nrow(data), n), ]
  })

  #Create plot
  output$plot <- renderPlot({
    data <- filtered_data()
    xvar <- input$xvar
    yvar <- input$yvar
    color_var <- if(input$color == "None") NULL else input$color

    if(input$plotType == "scatter") {
      ggplot(data, aes_string(x = xvar, y = yvar, color =
color_var)) +
        geom_point(size = 3, alpha = 0.7) +
        theme_minimal() +
        labs(title = paste("Scatter Plot:", xvar, "vs", yvar))

    } else if(input$plotType == "box") {
      ggplot(data, aes_string(x = color_var, y = xvar, fill =
color_var)) +
        geom_boxplot() +
        theme_minimal() +
        labs(title = paste("Box Plot of", xvar, "by",
color_var))

    } else if(input$plotType == "hist") {
      ggplot(data, aes_string(x = xvar, fill = color_var)) +
        geom_histogram(bins = 30, alpha = 0.7, position =
"identity") +
        theme_minimal() +
        labs(title = paste("Histogram of", xvar))
    }
  })

  #Show data table
  output$table <- renderDataTable({

```

```

    filtered_data()
  }, options = list(pageLength = 10))

  #Show summary
  output$summary <- renderPrint({
    summary(filtered_data())
  })

  #Download plot
  output$downloadPlot <- downloadHandler(
    filename = function() {
      paste("plot-", Sys.Date(), ".png", sep = "")
    },
    content = function(file) {
      ggsave(file, plot = output$plot(), device = "png",
        width = 10, height = 6)
    }
  )
}

#Run app
shinyApp(ui = ui, server = server)
```

```

## App 2: Simple Machine Learning Dashboard

```

```r
#ml_app.R
library(shiny)
library(caret)
library(ggplot2)
library(DT)

UI
ui <- fluidPage(
  titlePanel("Simple Machine Learning Dashboard"),

  sidebarLayout(
    sidebarPanel(
      width = 3,

      # File upload
      fileInput("file", "Upload CSV File",
        accept = c(".csv")),

```

```

#   Target variable
selectInput("target", "Target Variable:", choices = NULL),

#   Model selection
selectInput("model", "Select Model:",
            choices = c("Decision Tree" = "rpart",
                        "Random Forest" = "rf",
                        "Naive Bayes" = "nb",
                        "Logistic Regression" = "glm")),

#   Train/test split
sliderInput("split", "Train/Test Split (%):",
            min = 50, max = 90, value = 70, step = 5),

#   Action buttons
actionButton("train", "Train Model", class = "btn-
success"),
actionButton("reset", "Reset", class = "btn-warning")
),

mainPanel(
  width = 9,
  tabsetPanel(
    tabPanel("Data",
              h4("Data Preview"),
              dataTableOutput("preview"),
              h4("Summary"),
              verbatimTextOutput("dataSummary")),

    tabPanel("Model Results",
              h4("Confusion Matrix"),
              verbatimTextOutput("confusion"),
              h4("Performance Metrics"),
              tableOutput("metrics"),
              h4("Feature Importance"),
              plotOutput("importance")),

    tabPanel("Predictions",
              h4("Test Set Predictions"),
              dataTableOutput("predictions"),
              h4("Download Predictions"),
              downloadButton("downloadPreds", "Download
CSV"))

```

```

    )
  )
)

#Server
server <- function(input, output, session) {

  # Reactive values
  values <- reactiveValues(
    data = NULL,
    model = NULL,
    results = NULL
  )

  #Load data
  observeEvent(input$file, {
    req(input$file)
    values$data <- read.csv(input$file$datapath)

    # Update target variable choices
    updateSelectInput(session, "target",
                      choices = names(values$data))
  })

  #Reset
  observeEvent(input$reset, {
    values$data <- NULL
    values$model <- NULL
    values$results <- NULL
  })

  #Data preview
  output$preview <- renderDataTable({
    req(values$data)
    head(values$data, 20)
  })

  #Data summary
  output$dataSummary <- renderPrint({
    req(values$data)
    cat("Dataset Dimensions:", dim(values$data), "\n\n")
    cat("Column Names:\n")
    print(names(values$data))
  })
}

```

```

    cat("\n\nSummary:\n")
    print(summary(values$data))
  })

  #Train model
  observeEvent(input$train, {
    req(values$data, input$target)

    # Prepare data
    df <- values$data
    target_col <- input$target

    Ensure target is factor
    df[[target_col]] <- as.factor(df[[target_col]])

    #Split data
    set.seed(123)
    train_idx <- createDataPartition(df[[target_col]],
                                      p = input$split/100,
                                      list = FALSE)

    train_data <- df[train_idx, ]
    test_data <- df[-train_idx, ]

    #Train model
    withProgress(
      message = 'Training model...',
      detail = 'This may take a moment...',
      value = 0.5,
      {
        formula <- as.formula(paste(target_col, "~ ."))

        model <- train(
          formula,
          data = train_data,
          method = input$model,
          trControl = trainControl(method = "cv", number = 5)
        )

        # Make predictions
        predictions <- predict(model, test_data)

        # Calculate metrics
        cm <- confusionMatrix(predictions,
test_data[[target_col]])

```

```

# Store results
values$model <- model
values$results <- list(
  confusion = cm,
  predictions = data.frame(
    Actual = test_data[[target_col]],
    Predicted = predictions,
    Correct = test_data[[target_col]] == predictions
  ),
  test_data = test_data
)

incProgress(0.5, detail = "Finalizing...")
}
)

showNotification("Model training complete!", type =
"success")
})

# Confusion matrix
output$confusion <- renderPrint({
  req(values$results)
  print(values$results$confusion$table)
})

#Metrics table
output$metrics <- renderTable({
  req(values$results)
  cm <- values$results$confusion

  data.frame(
    Metric = c("Accuracy", "Kappa", "Sensitivity",
"Specificity"),
    Value = round(c(cm$overall["Accuracy"],
                    cm$overall["Kappa"],
                    cm$byClass["Sensitivity"],
                    cm$byClass["Specificity"]), 3)
  )
})

#Feature importance plot
output$importance <- renderPlot({

```

```

req(values$model)

# Get variable importance
imp <- varImp(values$model)$importance

if(!is.null(imp)) {
  imp_df <- data.frame(
    Feature = rownames(imp),
    Importance = imp$Overall
  )

  imp_df <- imp_df[order(imp_df$Importance, decreasing =
TRUE), ]
  imp_df$Feature <- factor(imp_df$Feature, levels =
imp_df$Feature)

  ggplot(imp_df[1:10, ], aes(x = Importance, y =
reorder(Feature, Importance))) +
    geom_bar(stat = "identity", fill = "steelblue") +
    theme_minimal() +
    labs(title = "Top 10 Important Features",
         x = "Importance", y = "Feature")
}
})

#Predictions table
output$predictions <- renderDataTable({
  req(values$results)
  values$results$predictions
})

# Download predictions
output$downloadPreds <- downloadHandler(
  filename = function() {
    paste("predictions-", Sys.Date(), ".csv", sep = "")
  },
  content = function(file) {
    write.csv(values$results$predictions, file, row.names =
FALSE)
  }
)
}

#Run app

```



```
shinyApp(ui = ui, server = server)
````
```

## 7. Best Practices

### Code Organization

```
````r
1. Modularize your code
source("helpers.R")    Helper functions
source("modules.R")    Shiny modules

2. Use modules for reusable components
myModuleUI <- function(id) {
  ns <- NS(id)
  UI code here
}

myModuleServer <- function(id) {
  moduleServer(id, function(input, output, session) {
    Server code here
  })
}

3. Separate data processing
process_data <- function(raw_data) {
  Clean and transform data
  cleaned <- na.omit(raw_data)
  More processing...
  return(cleaned)
}
````
```

### Performance Tips

```
````r
1. Use reactive expressions for expensive calculations
expensive_calc <- reactive({
  This only recalculates when inputs change
  calculate_something(input$param)
})
```

```

2. Debounce reactive inputs
debounced_input <- debounce(input$text, 500)    Wait 500ms

3. Use progress indicators
withProgress({
  Long-running code
}, message = "Processing...")
````

```

## Error Handling

```

````r
Use tryCatch for graceful error handling
output$result <- renderPrint({
  tryCatch({
    Potentially failing code
    result <- 10 / input$number
    paste("Result:", result)
  }, error = function(e) {
    paste("Error:", e$message)
  })
})

Validate inputs
validate(
  need(input$file, "Please upload a file"),
  need(input$threshold > 0, "Threshold must be positive")
)
````

```

## Styling

```

````r
Add CSS
ui <- fluidPage(
  tags$head(
    tags$style(HTML("
      .btn-primary {
        background-color: 3498db;
        border-color: 2980b9;
      }
      .shiny-output-error {
        color: red;
        font-weight: bold;
      }
    "))
  )
)

```

```

    }
    ") )
  ),
  Rest of UI
)
```

Testing
```r
Test reactive behavior
testServer(moduleServer, {
  Set inputs
  session$setInputs(x = 1)

  Check outputs
  expect_equal(output$result, 2)
})
```

```

## Quick Reference Card

### Common Inputs

```

```r
actionButton()      Clickable button
checkboxInput()        Checkbox
dateInput()          Date picker
fileInput()          File upload
numericInput()       Number input
radioButtons()       Radio buttons
selectInput()        Dropdown
sliderInput()        Slider
textInput()          Text box
```

```

### Common Outputs

```

```r
dataTableOutput()    Interactive table
htmlOutput()         HTML content
imageOutput()        Image
plotOutput()         Plot
tableOutput()        Simple table
```

```

```
textOutput() Text
verbatimTextOutput() Code/text output
```
```

Common Render Functions

```
```r
renderDataTable() Render data table
renderImage() Render image
renderPlot() Render plot
renderPrint() Render printed output
renderTable() Render table
renderText() Render text
renderUI() Render UI
```
```

Key Functions

```
```r
reactive() Create reactive expression
observe() Create observer
observeEvent() Observe specific event
eventReactive() Event-triggered reactive
reactiveValues() Create reactive values
isolate() Read value without dependency
```
```

Sample Data Files

sample_sales.csv

```
```csv
Date,Product,Region,Quantity,Revenue
2024-01-01,Product A,North,10,5000
2024-01-01,Product B,South,5,2500
2024-01-02,Product A,East,8,4000
2024-01-02,Product C,West,12,6000
2024-01-03,Product B,North,6,3000
2024-01-03,Product A,South,15,7500
2024-01-04,Product C,East,9,4500
2024-01-04,Product B,West,7,3500
2024-01-05,Product A,North,11,5500
```
```

```
2024-01-05,Product C,South,13,6500
```
```

customer\_data.csv

```
```csv
CustomerID,Age,Gender,Income,Score,Segment
1001,34,Male,55000,85,Premium
1002,45,Female,62000,92,Premium
1003,28,Male,42000,78,Standard
1004,52,Female,71000,88,Premium
1005,39,Male,48000,81,Standard
1006,61,Female,58000,79,Standard
1007,32,Male,53000,90,Premium
1008,47,Female,67000,87,Premium
1009,29,Male,44000,76,Standard
1010,55,Female,69000,89,Premium
```
```

ml\_classification.csv

```
```csv
feature1,feature2,feature3,label
5.1,3.5,1.4,0
4.9,3.0,1.4,0
4.7,3.2,1.3,0
4.6,3.1,1.5,0
5.0,3.6,1.4,0
7.0,3.2,4.7,1
6.4,3.2,4.5,1
6.9,3.1,4.9,1
5.5,2.3,4.0,1
6.5,2.8,4.6,1
```
```